

新版 MG SDK 配置页面接口设计

一、统一约定

1. 接口前缀

MG 前端调用：

代码块

```
1 /customer/topsdk/console/meetgames/sdk-config
```

gp-sdk Controller：

代码块

```
1 /meetgames/sdk-config
```

2. HTTP 方法

- GET：查询。
- POST：创建和局部保存 JSON 配置。
- PUT：替换或更新一个已经明确定位的配置资源。本方案使用 PUT 上传并更新当前游戏的 Google 支付配置文件或 Firebase 平台配置文件。

3. ID 类型

appId、channelId、strategyId、fileId 在接口中统一使用 String，避免前端 JavaScript 丢失 BIGINT 精度。

4. 统一响应

代码块

```
1  {
2    "code": 200,
3    "message": "",
4    "result": {},
5    "traceId": "sdk-console-server,..."
6  }
```

code=200 或 code=0 表示成功，业务数据放在 result。

5. 登录类型约定

所有与登录方式相关的入参和出参均不传登录方式英文名称，统一使用 ThirdPlatformType.code 的数字值。为避免前端 JavaScript 数字类型差异，接口中序列化为数字字符串。

ThirdPlatformType	接口 type
GUEST	"1"
FACEBOOK	"2"
GOOGLE	"3"
SNAPCHAT	"5"
TWITTER	"6"
APPLE	"9"
NAVER	"11"

KAKAO	"12"
LINE	"13"
EMAIL	"21"
DISCORD	"26"
TIKTOK	"27"

具体登录方式展示名称由前端根据 `ThirdPlatformType` 对照表处理。后端校验和保存时将数字字符串转换成对应的 `ThirdPlatformType.code`。

二、配置中心接口

1. 查询模块配置总览

进入配置中心首页时调用。只返回各模块的配置状态和配置摘要，不返回完整参数和密钥。

代码块

```
1 GET /getModuleConfigOverview?appId={appId}
```

完整地址：

代码块

```
1 GET /customer/topsdk/console/meetgames/sdk-config/getModuleConfigOverview
```

入参

参数	位置	类型	必传	说明
appId	Query	String	是	当前游戏 ID

出参

代码块

```
1  {
2    "appId": "791251136341225472",
3    "modules": [
4      {
5        "moduleCode": "payment",
6        "moduleName": "支付",
7        "configured": true,
8        "items": [
9          {
10           "code": "apple",
11           "name": "App Store",
12           "selected": true,
13           "configured": true
14         },
15         {
16           "code": "google",
17           "name": "Google Play",
18           "selected": true,
19           "configured": true
20         }
21       ]
22     },
23     {
24       "moduleCode": "login",
25       "moduleName": "登录",
26       "configured": true,
27       "items": [
28         {
29           "type": "1",
30           "selected": true,
31           "configured": true
```

```
32     },
33     {
34         "type": "3",
35         "selected": true,
36         "configured": true
37     },
38     {
39         "type": "2",
40         "selected": false,
41         "configured": true
42     }
43 ]
44 },
45 {
46     "moduleCode": "agreement",
47     "moduleName": "协议与隐私",
48     "configured": true,
49     "items": [
50         {
51             "code": "agreement-default",
52             "name": "默认协议分组",
53             "configured": true
54         },
55         {
56             "code": "agreement-eu",
57             "name": "欧洲协议分组",
58             "configured": true
59         }
60     ]
61 },
62 {
63     "moduleCode": "compliance",
64     "moduleName": "合规",
65     "configured": true,
66     "items": [
67         {
68             "code": "kws",
69             "name": "KWS",
70             "configured": true
71         },
72         {
73             "code": "ageThreshold",
74             "name": "年龄阈值",
75             "configured": true,
76             "value": 13
77         }
78     ]
79 }
```

```
79     },
80     {
81       "moduleCode": "thirdPartyData",
82       "moduleName": "三方数据",
83       "configured": true,
84       "items": [
85         {
86           "code": "firebase",
87           "name": "Firebase",
88           "configured": true
89         },
90         {
91           "code": "appsflyer",
92           "name": "AppsFlyer",
93           "configured": true
94         }
95       ]
96     },
97     {
98       "moduleCode": "advertising",
99       "moduleName": "广告变现",
100       "configured": true,
101       "items": [
102         {
103           "code": "applovin",
104           "name": "AppLovin MAX",
105           "configured": true
106         }
107       ]
108     },
109     {
110       "moduleCode": "customerService",
111       "moduleName": "客服工具",
112       "configured": true,
113       "items": []
114     }
115   ]
116 }
```

出参字段

字段	类型	说明
<code>moduleCode</code>	String	模块编码
<code>moduleName</code>	String	模块中文名称
<code>configured</code>	Boolean	模块当前是否满足完整配置要求
<code>items</code>	Array	模块已经配置过的功能摘要
<code>items[].code</code>	String	功能编码
<code>items[].name</code>	String	页面展示名称
<code>items[].type</code>	String	登录方式专用，值为 <code>ThirdPlatformType.code</code> 数字字符串；登录摘要不返回名称
<code>items[].selected</code>	Boolean	当前是否选中，主要用于登录方式
<code>items[].configured</code>	Boolean	该功能的必填参数是否完整
<code>items[].value</code>	Object	总览需要展示的简单值

各模块摘要内容

模块	<code>items</code> 返回内容
<code>payment</code>	App Store、Google Play、ONE Store 等支付渠道
<code>login</code>	<code>ThirdPlatformType.code</code> 、是否选中、参数是否完整，不返回登录方式名称
<code>agreement</code>	协议分组 ID 和分组名称
<code>compliance</code>	KWS 是否完整、年龄阈值

thirdPartyData	Firebase、AppsFlyer、Adjust
advertising	当前广告平台名称
customerService	使用旧客服服务判断是否配置

客服工具保存仍使用原客服接口。

2. 查询单个模块详细配置

点击模块进入配置弹窗或详情页时调用。

代码块

```
1 GET /getModuleConfigDetail?appId={appId}&moduleCode={moduleCode}
```

入参

参数	位置	类型	必传	说明
appId	Query	String	是	游戏 ID
moduleCode	Query	String	是	模块编码

moduleCode 支持：

代码块

- 1 payment
- 2 login
- 3 agreement
- 4 compliance
- 5 thirdPartyData
- 6 advertising

客服详情继续调用原客服接口。

通用出参

代码块

```
1  {
2    "appId": "791251136341225472",
3    "moduleCode": "login",
4    "schemaVersion": 1,
5    "version": 3,
6    "configured": true,
7    "config": {},
8    "missingFields": [],
9    "updatedAt": "2026-08-12 10:00:00"
10 }
```

字段	类型	说明
appId	String	游戏 ID
moduleCode	String	模块编码
schemaVersion	Integer	当前 JSON 结构版本
version	Long	当前配置修改版本
configured	Boolean	模块是否完整
config	Object	模块完整配置
missingFields	String[]	尚未填写的必填字段路径

updatedAt	String	最后更新时间
-----------	--------	--------

详情接口中的 Secret、Token、License Key 等敏感参数按实际保存值明文返回，不再返回掩码字符串。Google 支付是例外：由于页面只有文件上传交互，其 `pay_secret` 不返回，支付详情只返回 `uploadStatus` 和 `fileName`。接口必须通过应用权限校验并仅允许在 HTTPS 管理后台调用，同时禁止在访问日志和操作日志中记录响应体。

Google 支付和 Firebase 配置文件均通过各自的专用 `PUT` 接口上传，并由上传接口直接更新对应模块配置。

3. 查询登录方式参数配置项

该接口用于获取当前服务端支持的登录方式，以及每种登录方式保存配置时使用的参数 key 和页面展示标题。功能与现有 `getChannelParameter` 相近，但返回结构固定拆分为公共参数、iOS 参数和 Android 参数。

代码块

```
1 GET /getLoginParameterConfigs?type={type}
```

完整地址：

代码块

```
1 GET /customer/topsdk/console/meetgames/sdk-config/getLoginParameterConfigs
```

参数	位置	类型	必传	说明
type	Query	String	否	ThirdPlatformType.code 数字字符串。传入时只查询对应登录方式；不传时返回全部支持的登录方式

查询单个登录方式：

代码块

```
1 GET /getLoginParameterConfigs?type=3
```

查询全部登录方式：

代码块

```
1 GET /getLoginParameterConfigs
```

无论查询单个还是全部，业务数据 result 始终为 JSON 数组。查询单个时返回长度为 1 的数组；传入不支持的 type 时返回参数错误。

出参

代码块

```
1 [
2   {
3     "type": "3",
4     "params": {
5       "common": {
6         "webClientId": "Web客户端ID",
7         "clientSecret": "客户端密钥"
```

```
8      },
9      "ios": {
10         "iosClientId": "iOS客户端ID",
11         "iosUrlScheme": "iOS应用跳转Scheme",
12         "iosBundleId": "iOS Bundle ID"
13     },
14     "android": {
15         "androidClientId": "Android客户端ID",
16         "androidPackageName": "Android包名",
17         "androidSha1": "Android签名SHA-1"
18     }
19 }
20 }
21 ]
```

字段	类型	说明
type	String	登录类型，值为 ThirdPlatformType.code 数字字符串
params.common	Object	公共参数定义，key 是保存配置时传入的参数名，value 是页面展示标题
params.ios	Object	iOS 特有参数定义
params.android	Object	Android 特有参数定义

没有参数的作用域也必须返回空对象，例如 Guest：

代码块

```
1  [
2  {
3      "type": "1",
4      "params": {
5          "common": {},
6          "ios": {},
7          "android": {}
8      }
9  }
```

当前支持的参数定义

接口支持的登录类型以 `ThirdPlatformType` 中 `authName != null` 的类型为准。下表使用“保存 key：页面标题”表示返回内容：

type	common	ios	android
"1"	无	无	无
"21"	无	无	无
"9"	teamId: Team ID、 keyId: Key ID、 privateKey: Private Key	iosBundleId: iOS Bundle ID	androidServicesId: Android Services ID、 androidRedirectUri: Android回调地址
"3"	webClientId: Web客户端 ID、 clientSecret: 客户端密钥	iosClientId: iOS客户端 ID、 iosUrlScheme: iOS应用跳转Scheme、 iosBundleId: iOS Bundle ID	androidClientId: Android客户端ID、 androidPackageName: Android包名、 androidSha1: Android签名SHA-1
"12"	nativeAppKey: Native App Key	iosBundleId: iOS Bundle ID、 iosUrlScheme: iOS应用跳转Scheme	androidPackageName: Android包名、 androidKeyHash: Android Key Hash
"2"	appId: App ID、 clientToken: Client Token	iosBundleId: iOS Bundle ID、 iosUrlScheme: iOS应用跳转Scheme	androidPackageName: Android包名、 androidKeyHash: Android Key Hash
"6"	apiKey: API Key、 apiSecret: API Secret、 callbackUrl: 回调地址	无	无
"5"	clientId: OAuth Client ID、 redirectUri: 回调地址	iosBundleId: iOS Bundle ID	androidPackageName: Android包名

	址		
"13"	channelId: Channel ID、channelSecret: Channel Secret	iosBundleId: iOS Bundle ID、iosUniversalLink: iOS Universal Link	androidPackageName: Android包名
"11"	clientId: Client ID、clientSecret: Client Secret	iosUrlScheme: iOS应用跳转Scheme	androidPackageName: Android包名
"27"	clientKey: Client Key、clientSecret: Client Secret	iosUrlScheme: iOS应用跳转Scheme	androidRedirectScheme: Android回跳Scheme、androidPackageName: Android包名
"26"	clientId: Client ID、clientSecret: Client Secret、redirectUri: 回调地址	无	无

页面不能自行写死参数标题；新增登录类型或参数时，由后端更新该接口的参数定义，模块保存校验器使用同一份定义。

4. 局部保存单个模块

代码块

```
1 POST /saveModuleConfig
2 Content-Type: application/json
```

通用入参

代码块

```
1  {
2    "appId": "791251136341225472",
3    "moduleCode": "login",
4    "config": {}
5  }
```

参数	类型	必传	说明
appId	String	是	游戏 ID
moduleCode	String	是	模块编码，用于确定存储、校验和渠道同步逻辑
config	Object	是	本次需要修改的字段，不要求提交完整模块 JSON

通用出参

代码块

```
1  {
2    "moduleCode": "login",
3    "version": 4,
4    "configured": true,
5    "config": {},
6    "missingFields": [],
7    "items": [],
8    "syncedChannelCount": 3,
9    "updatedAt": "2026-08-12 10:10:00"
10 }
```

字段	类型	说明
version	Long	保存后的版本
configured	Boolean	

		合并后的完整配置是否满足要求
config	Object	合并后的完整模块配置；支付模块不返回 channels.google.pay_secret 和文件地址，只返回 Google 上传状态与文件名
missingFields	String[]	仍缺少的必填参数
items	Array	保存后的模块摘要
syncedChannelCount	Integer	本次同步的渠道数量
updatedAt	String	保存时间

局部更新规则

- 未传字段：保持原值。
- 普通对象：递归合并。
- null：清除已有字段。
- 空字符串：明确保存为空字符串。
- 登录数组按 type 合并。
- 协议分组按 groupId 合并。
- 三方数据按 type 合并，但 type=firebase 不允许通过本接口保存，必须调用 Firebase 专用 PUT 上传接口。
- 缺少必填参数允许保存，但返回 configured=false。
- 未知字段、类型错误和格式错误拒绝保存。

三、各模块参数

1. 支付 payment

详情接口返回的 config

代码块

```
1  {
2    "notifyUrl": "https://api.example.com/payment/notify",
3    "channels": {
4      "apple": {
5        "pay_secret": "apple-shared-secret"
6      },
7      "google": {
8        "uploadStatus": "已上传",
9        "fileName": "google-payment-config.json"
10     },
11     "oneStore": {
12       "licenseKey": "one-store-license-key",
13       "clientId": "one-store-client-id"
14     }
15   }
16 }
```

参数说明

参数	类型	说明
notifyUrl	String	支付成功后的服务端发货地址
channels	Object	支付渠道配置
channels.apple.pay_secret	String	Apple 支付共享密钥
channels.google.uploadStatus	String	仅详情返回；已配置时固定为 已上传 ，未配置时为空字符串

<code>channels.google.fileName</code>	String	仅详情返回；当前 Google 支付配置文件名，未配置时为空字符串
<code>channels.oneStore.licenseKey</code>	String	ONE Store License Key
<code>channels.oneStore.clientId</code>	String	ONE Store Client ID

保存支付模块的入参

调用 `POST /saveModuleConfig` 且 `moduleCode=payment` 时，`config` 只接收以下支付参数：

代码块

```
1  {
2    "notifyUrl": "https://api.example.com/payment/notify",
3    "channels": {
4      "apple": {
5        "pay_secret": "apple-shared-secret"
6      },
7      "oneStore": {
8        "licenseKey": "one-store-license-key",
9        "clientId": "one-store-client-id"
10     }
11   }
12 }
```

- `POST /saveModuleConfig` 不接收任何 `channels.google` 入参；请求中出现该字段时返回参数错误，并提示使用 Google 支付配置文件上传接口。
- Google 支付的上传、文件解析、`pay_secret` 存储和渠道同步全部由专用 `PUT /uploadGooglePaymentConfig` 接口完成。
- Apple 仍通过 `channels.apple.pay_secret` 保存共享密钥；Google 的 `pay_secret` 只 在后端存储，不返回给前端。

2. 登录 login

详情/保存的 config

代码块

```
1  {
2    "methods": [
3      {
4        "type": "1",
5        "selected": true,
6        "configured": true,
7        "params": {}
8      },
9      {
10       "type": "3",
11       "selected": true,
12       "configured": true,
13       "params": {
14         "common": {
15           "webClientId": "client-id",
16           "clientSecret": "google-client-secret"
17         },
18         "ios": {
19           "iosClientId": "ios-client-id",
20           "iosUrlScheme": "com.googleusercontent.apps.xxx",
21           "iosBundleId": "com.company.game"
22         },
23         "android": {
24           "androidClientId": "android-client-id",
25           "androidPackageName": "com.company.game",
26           "androidSha1": "12:34:..."
27         }
28       }
29     ]
30   }
31 }
```

参数说明

参数	类型	说明
methods	Array	已经保存过的登录方式
methods[].type	String	登录方式唯一编码，使用ThirdPlatformType.code 数字字符串
methods[].selected	Boolean	当前是否选中
methods[].configured	Boolean	出参字段，由后端计算参数是否完整
methods[].params	Object	登录参数
params.common	Object	通用参数
params.ios	Object	iOS 参数
params.android	Object	Android 参数

只修改选中状态

代码块

```
1  {
2    "appId": "791251136341225472",
3    "moduleCode": "login",
4    "config": {
5      "methods": [
6        {
7          "type": "3",
8          "selected": false
9        }
10     ]
11   }
12 }
```

处理规则：

- 只修改 `selected=false`。
- 原 `params` 完整保留。
- 同步渠道时只将 `channel_auth.enable` 改为 `false`。
- `clientId`、`secret`、`redirect` 等参数不清空。
- 再次选中时无需重新填写。

登录模块整体完整条件：

- 至少有一种登录方式被选中。
- 所有被选中的登录方式参数完整。
- 未选中的登录方式参数不完整，不影响模块整体状态。

3. 协议 `agreement`

代码块

```
1  {
2    "groups": [
3      {
4        "groupId": "agreement-default",
5        "name": "默认协议分组",
6        "defaultLanguage": "en",
7        "locales": {
8          "en": {
9            "name": "Privacy Policy",
10           "url": "https://example.com/privacy/en"
11         },
12         "zh-CN": {
13           "name": "隐私政策",
14           "url": "https://example.com/privacy/zh-cn"
15         }
16       }
17     ]
18   }
```

```
17     }
18   ]
19 }
```

参数	类型	说明
groups	Array	协议分组
groupId	String	分组 ID；新增时可不传，由后端生成
name	String	协议分组名称
defaultLanguage	String	默认语言编码
locales	Object	多语言协议内容
locales. {language}.name	String	协议名称
locales. {language}.url	String	协议地址

删除分组：

```
代码块
1  {
2    "groupId": "agreement-default",
3    "_delete": true
4  }
```

4. 合规 compliance

合规只配置 KWS 和年龄阈值。

代码块

```
1  {
2    "ageThreshold": 13,
3    "clientId": "kws-client",
4    "clientSecret": "kws-client-secret",
5    "verifySecret": "kws-verify-secret",
6    "redirectUrl": "https://api.example.com/kws/callback"
7  }
```

参数	类型	说明
ageThreshold	Integer	年龄阈值，范围 0~18
clientId	String	KWS Client ID
clientSecret	String	KWS Client Secret
verifySecret	String	KWS 回调验证密钥
redirectUrl	String	KWS 回调地址

数据写入 app_parent_config，不写入 sdk_module_config。

保存年龄阈值后：

- 同步当前 appId 下所有渠道的 channel.age_threshold。
- 没有渠道时不创建渠道。
- GDPR 和年龄家长控制开关不在此接口维护。
- 新渠道相关开关继续使用默认关闭状态。

Secret 字段规则：

- 详情接口返回数据库中实际保存的明文。
- 保存时不传：不修改。
- 传入明文新值：更新。
- 传 `null`：清空。

5. 三方数据 `thirdPartyData`

详情接口返回的 `config`

代码块

```
1  {
2    "platforms": [
3      {
4        "type": "firebase",
5        "config": {
6          "androidFile": {
7            "fileName": "google-services.json",
8            "fileUrl": "https://bucket.example.com/sdk/google-services.json"
9          },
10         "iosFile": {
11           "fileName": "GoogleService-Info.plist",
12           "fileUrl": "https://bucket.example.com/sdk/GoogleService-Info.plist"
13         }
14       }
15     ],
16     {
17       "type": "appsflyer",
18       "config": {
19         "devKey": "appsflyer-dev-key",
20         "appleAppId": "6748123091"
21       }
22     },
23     {
24       "type": "adjust",
25       "config": {
26         "appIdentifier": "adjust-app-token",
```



```
27         "loginEvent": "event-id",
28         "signUpEvent": "event-id",
29         "tutorialBeginEvent": "event-id",
30         "tutorialCompleteEvent": "event-id",
31         "levelUpEvent": "event-id",
32         "achievementEvent": "event-id",
33         "purchaseEvent": "event-id",
34         "shareEvent": "event-id",
35         "earnCurrencyEvent": "event-id",
36         "spendCurrencyEvent": "event-id",
37         "levelStartEvent": "event-id",
38         "levelEndEvent": "event-id"
39     }
40 }
41 ]
42 }
```

参数	类型	说明
platforms[].type	String	firebase/appsflyer/adjust
platforms[].config	Object	平台参数
androidFile/iosFile.fileName	String	Firebase 对应平台当前上传的文件名；未上传时返回空字符串
androidFile/iosFile.fileUrl	String	Firebase 对应平台文件上传后的地址；未上传时返回空字符串
devKey	String	AppsFlyer Dev Key
appleAppId	String	Apple App ID
appIdentifier	String	Adjust App Token
*Event	String	Adjust 标准事件 ID

保存三方数据模块的入参

调用 `POST /saveModuleConfig` 且 `moduleCode=thirdPartyData` 时，`config.platforms` 只接收无需上传文件的平台参数，例如：

代码块

```
1  {
2    "platforms": [
3      {
4        "type": "appsflyer",
5        "config": {
6          "devKey": "appsflyer-dev-key",
7          "appleAppId": "6748123091"
8        }
9      },
10     {
11       "type": "adjust",
12       "config": {
13         "appIdentifier": "adjust-app-token",
14         "loginEvent": "event-id",
15         "purchaseEvent": "event-id"
16       }
17     }
18   ]
19 }
```

- `POST /saveModuleConfig` 不接收 `type=firebase`，也不接收 `androidFile`、`iosFile`、`fileName`、`fileUrl` 或文件内容。
- Firebase Android 和 iOS 文件分别通过专用 `PUT /uploadFirebaseConfig` 接口上传并保存。
- 三方数据详情和保存后的 `config` 可以返回 Firebase 的 `fileName`、`fileUrl`，但不返回内部文件记录 ID。

6. 广告变现 `advertising`

代码块

```
1  {
2    "provider": "MAX",
3    "config": {
4      "admobAppId": "ca-app-pub-xxx",
5      "maxSdkKey": "applovin-max-sdk-key"
6    }
7  }
```

参数	类型	说明
provider	String	NONE、ADMOB、MAX
config.admobAppId	String	AdMob App ID
config.maxSdkKey	String	AppLovin MAX SDK Key

完整性规则：

- NONE：表示未启用广告。
- ADMOB：要求 admobAppId。
- MAX：要求 admobAppId 和 maxSdkKey。

四、文件上传接口

1. 上传并更新 Google 支付配置

Google 支付在前端只有上传 JSON 文件的交互，因此由该接口一次完成文件上传、内容解析和 Google 支付配置存储，不再调用 `POST /saveModuleConfig` 保存 Google 参数。

代码块

```
1 PUT /uploadGooglePaymentConfig
2 Content-Type: multipart/form-data
```

完整地址：

代码块

```
1 PUT /customer/topsdk/console/meetgames/sdk-config/uploadGooglePaymentConfig
```

`PUT` 表示替换当前 `appId` 已有的 Google 支付配置；首次上传时创建，重复上传时以新文件内容覆盖当前配置。

入参

参数	位置	类型	必传	说明
<code>appId</code>	Form Data	String	是	游戏 ID
<code>file</code>	Form Data	MultipartFile	是	Google 支付 JSON 配置文件，仅接受 <code>.json</code>

接口不接收 `moduleCode`、`usageCode`、`pay_secret`、`fileId` 或其他 Google 支付参数；模块和文件用途由接口固定为 `payment`、`GOOGLE_PAYMENT_SERVER_CONFIG`。

服务端处理

代码块

- 1 校验appId及当前用户的应用权限
- 2 → 校验扩展名、MIME类型、文件大小
- 3 → 上传文件到对象存储并取得fileUrl
- 4 → 上传成功后按UTF-8读取文件内容
- 5 → 解析并校验文件是合法JSON
- 6 → 不提取具体字段，将完整JSON保存为jsonString
- 7 → 事务写入sdk_config_file和sdk_module_config
- 8 → appId已有渠道时同步更新渠道Google支付配置
- 9 → 返回上传状态和文件名

存储规则：

- sdk_config_file 保存文件名、objectKey、fileUrl、摘要等文件信息，满足上传文件地址必须入库的要求。
- sdk_module_config.config_json 的 payment.channels.google.pay_secret 保存完整 JSON 字符串。
- payment.channels.google.serverConfigFile 保存 fileId、fileName 和 fileUrl，用于后台追溯文件来源。
- 解析只校验 JSON 格式，不读取或校验 type、project_id、private_key 等具体字段。
- 若当前 appId 尚未创建渠道，只更新模块配置；后续创建渠道时再同步 Google 支付配置。
- 若已经创建渠道，在模块配置保存成功后同步更新现有渠道的 Google 支付配置。

后端最终存储结构示例：

代码块

```
1  {
2    "channels": {
3      "google": {
4        "pay_secret": "{\"type\":\"service_account\",\"project_id\":\"demo-project\"}",
5        "serverConfigFile": {
6          "fileId": "9000001",
7          "fileName": "google-payment-config.json",
8          "fileUrl": "https://bucket.example.com/sdk/google-payment-config.json"
9        }
10   }
11 }
```

```
12 }
```

上传文件成功，但后续读取、JSON 解析、数据库保存或渠道同步前的核心事务失败时，需要删除本次上传的对象文件并保留原 Google 支付配置，不允许出现文件已替换但配置未更新的状态。

`pay_secret` 和 `fileUrl` 均不得在该接口响应、访问日志、操作日志或异常日志中输出。

出参

统一响应中的 `result` 只返回上传状态和文件名：

代码块

```
1 {  
2   "uploadStatus": "已上传",  
3   "fileName": "google-payment-config.json"  
4 }
```

字段	类型	说明
<code>uploadStatus</code>	String	上传并保存成功后固定返回 <code>已上传</code>
<code>fileName</code>	String	本次上传的原文件名

不返回 `fileId`、`fileUrl`、`objectKey`、`pay_secret` 或解析后的 JSON 内容。

2. 上传并更新 Firebase 配置文件

Firebase 在三方数据页面只有 Android、iOS 两个文件上传交互。该接口一次上传一个平台的配置文件，并直接更新 `thirdPartyData` 模块中对应平台的文件名和文件地址，不再通过 `POST /saveModuleConfig` 保存 Firebase 参数。

代码块


```

1  PUT /uploadFirebaseConfig
2  Content-Type: multipart/form-data
        
```

完整地址：

代码块


```

1  PUT /customer/topsdk/console/meetgames/sdk-config/uploadFirebaseConfig
        
```

`PUT` 表示替换当前 `appId + platform` 对应的 Firebase 配置文件；Android 与 iOS 配置互不覆盖。首次上传时创建，再次上传同一平台时替换该平台原配置。

入参

参数	位置	类型	必传	说明
appId	Form Data	String	是	游戏 ID
platform	Form Data	String	是	android 或 ios
file	Form Data	MultipartFile	是	Android 上传 .json ，iOS 上传 .plist

接口不接收 `moduleCode`、`type`、`androidFile`、`iosFile`、`fileName`、`fileUrl` 或 `fileId`；模块和三方平台由接口固定为 `thirdPartyData`、`firebase`。

服务端处理

代码块

- 1 校验appId及当前用户的应用权限
- 2 → 校验platform与文件扩展名是否匹配
- 3 → 上传文件到对象存储并取得fileUrl
- 4 → 将文件名和fileUrl写入sdk_config_file
- 5 → 事务更新sdk_module_config中的Firebase平台文件配置
- 6 → appId已有渠道时同步更新渠道Firebase配置
- 7 → 返回上传状态、平台、文件名和文件地址

该接口不解析 Firebase 配置文件中的具体参数，只保存上传后的文件名和文件地址。模块最终存储结构示例：

代码块

```
1  {
2    "platforms": [
3      {
4        "type": "firebase",
5        "config": {
6          "androidFile": {
7            "fileName": "google-services.json",
8            "fileUrl": "https://bucket.example.com/sdk/google-services.json"
9          },
10         "iosFile": {
11           "fileName": "GoogleService-Info.plist",
12           "fileUrl": "https://bucket.example.com/sdk/GoogleService-Info.plist"
13         }
14       }
15     ]
16   }
17 }
```


存储和更新规则：

- `platform=android` 只更新 `config.androidFile`，保持 `config.iosFile` 原值不变。
- `platform=ios` 只更新 `config.iosFile`，保持 `config.androidFile` 原值不变。
- `androidFile`、`iosFile` 在模块 JSON 中均只保存 `fileName` 和 `fileUrl`，不保存 `fileId`、`objectKey`、文件或解析参数。
- `sdk_config_file` 仍记录对象存储 Key、文件地址、摘要等内部文件信息，用于追溯和文件管理。
- 若当前 `appId` 尚未创建渠道，只更新模块配置；后续创建渠道时再同步 Firebase 配置。
- 若已经创建渠道，在模块配置保存成功后同步更新现有渠道的 Firebase 配置。
- 上传或数据库保存失败时保留原平台配置；若新对象文件已经上传，则删除本次上传的对象文件。

出参

统一响应中的 `result` 返回：

代码块

```
1 {
2   "uploadStatus": "已上传",
3   "platform": "android",
4   "fileName": "google-services.json",
5   "fileUrl": "https://bucket.example.com/sdk/google-services.json"
6 }
```

字段	类型	说明
<code>uploadStatus</code>	String	上传并保存成功后固定返回 <code>已上传</code>
<code>platform</code>	String	本次更新的平台， <code>android</code> 或 <code>ios</code>

fileName	String	本次上传的原文件名
fileUrl	String	本次文件上传后的地址

不返回 `fileId`、`objectKey`、文件或 Firebase 文件内的具体参数。

五、渠道包接口

1. 查询渠道列表

代码块

```
1 GET /getChannels?appId={appId}&pageNo=1&pageSize=20
```

入参

参数	类型	必传	说明
appId	String	是	游戏 ID
pageNo	Integer	是	页码，从 1 开始
pageSize	Integer	是	每页数量

出参

代码块

```
2    "total": 2,  
3    "list": [  
4      {  
5        "channelId": "818920530665046041",  
6        "channelType": "GOOGLE",  
7        "channelName": "Google Play",  
8        "devicePlatform": "android",  
9        "packageName": "com.company.game",  
10       "capabilities": [  
11         "login",  
12         "payment",  
13         "agreement"  
14       ],  
15       "configurationComplete": true,  
16       "updatedAt": "2026-08-12 11:00:00"  
17     }  
18   ]  
19 }
```

2. 创建渠道

代码块

```
1  POST /createChannel
```

入参

代码块

```
1  {  
2    "appId": "791251136341225472",  
3    "channelType": "GOOGLE",  
4    "packageName": "com.company.game",  
5    "capabilities": [  
6      "login",  
7      "payment",  
8      "agreement"
```

```
9     ]
10 }
```

参数	类型	必传	说明
appId	String	是	游戏 ID
channelType	String	是	APPLE/GOOGLE/ONESTORE
packageName	String	是	渠道包名
capabilities	String[]	是	当前渠道包包含的 SDK 能力

创建时：

1. 校验包名。
2. 校验所选能力对应模块已配置完整。
3. 创建 channel。
4. 读取 sdk_module_config 初始化旧渠道配置表。
5. 读取 app_parent_config.age_threshold。
6. 写入新渠道年龄阈值。
7. GDPR 和年龄家长控制开关使用渠道默认值 false。
8. 保存渠道能力。
9. 创建默认发行策略。

出参

```
代码块
1  {
2      "channelId": "818920530665046041",
```

```
3  "packageName": "com.company.game",
4  "capabilities": [
5    "login",
6    "payment",
7    "agreement"
8  ],
9  "createdAt": "2026-08-12 11:10:00"
10 }
```

3. 修改渠道

代码块

```
1  POST /updateChannel
```

入参

代码块

```
1  {
2    "appId": "791251136341225472",
3    "channelId": "818920530665046041",
4    "packageName": "com.company.game.new",
5    "capabilities": [
6      "login",
7      "payment",
8      "agreement",
9      "advertising"
10   ]
11 }
```

渠道类型不可修改。

出参

返回修改后的渠道完整信息，结构与渠道列表项一致。

六、运营配置接口

1. 查询运营配置

代码块

```
1 GET /getOperationConfig?appId={appId}&channelId={channelId}
```

入参

参数	类型	必传	说明
appId	String	是	游戏 ID
channelId	String	是	渠道 ID

出参

代码块

```
1 {
2   "appId": "791251136341225472",
3   "channelId": "818920530665046041",
4   "packageName": "com.company.game",
```

```
5  "capabilities": [  
6    "login",  
7    "payment",  
8    "agreement"  
9  ],  
10 "channelConfig": {  
11   "gdprCheck": false,  
12   "ageParentalCheck": false,  
13   "ageThreshold": 13  
14 },  
15 "availableLoginTypes": [  
16   "1",  
17   "3"  
18 ],  
19 "availableAgreementGroups": [  
20   {  
21     "groupId": "agreement-default",  
22     "name": "默认协议分组"  
23   }  
24 ],  
25 "strategies": [  
26   {  
27     "strategyId": "10001",  
28     "strategyName": "默认策略",  
29     "isDefault": true,  
30     "countryCodes": [],  
31     "config": {  
32       "enabledCapabilities": [  
33         "login",  
34         "payment"  
35       ],  
36       "loginMethods": [  
37         "1",  
38         "3"  
39       ],  
40       "agreementGroupIds": [  
41         "agreement-default"  
42       ]  
43     }  
44   },  
45   {  
46     "strategyId": "10002",  
47     "strategyName": "欧洲策略",  
48     "isDefault": false,  
49     "countryCodes": [  
50       "DE",  
51       "FR",
```

```
52         "IT",
53         "ES"
54     ],
55     "config": {
56         "enabledCapabilities": [
57             "login",
58             "payment",
59             "agreement"
60         ],
61         "loginMethods": [
62             "1",
63             "21",
64             "3"
65         ],
66         "agreementGroupIds": [
67             "agreement-default"
68         ]
69     }
70 }
71 ]
72 }
```

2. 保存运营配置

代码块

```
1  POST /saveOperationConfig
```

入参

代码块

```
1  {
2      "appId": "791251136341225472",
3      "channelId": "818920530665046041",
4      "channelConfig": {
5          "gdprCheck": true,
6          "ageParentalCheck": false
```



```
7  },
8  "strategies": [
9    {
10     "strategyId": "10001",
11     "strategyName": "默认策略",
12     "isDefault": true,
13     "countryCodes": [],
14     "config": {
15       "enabledCapabilities": [
16         "login",
17         "payment"
18       ],
19       "loginMethods": [
20         "1",
21         "3"
22       ],
23       "agreementGroupIds": [
24         "agreement-default"
25       ]
26     }
27   },
28   {
29     "strategyName": "欧洲策略",
30     "isDefault": false,
31     "countryCodes": [
32       "DE",
33       "FR"
34     ],
35     "config": {
36       "enabledCapabilities": [
37         "login",
38         "payment",
39         "agreement"
40       ],
41       "loginMethods": [
42         "1",
43         "21",
44         "3"
45       ],
46       "agreementGroupIds": [
47         "agreement-default"
48       ]
49     }
50   }
51 ]
52 }
```

入参说明

参数	类型	说明
channelConfig.gdprCheck	Boolean	渠道 GDPR 总开关
channelConfig.ageParentalCheck	Boolean	渠道年龄与家长控制总开关
strategies	Array	当前渠道的完整发行策略列表
strategyId	String	已存在策略必传；新增策略不传
strategyName	String	策略名称
isDefault	Boolean	是否默认兜底策略
countryCodes	String[]	策略适用国家数组
enabledCapabilities	String[]	当前策略启用的 SDK 能力
loginMethods	String[]	登录方式及顺序；元素为 ThirdPlatformType.code 数字字符串
agreementGroupIds	String[]	协议分组 ID

出参

代码块

```
1  {
2    "channelId": "818920530665046041",
3    "strategies": [],
4    "updatedAt": "2026-08-12 12:00:00"
5  }
```

保存规则：

- 每个渠道必须且只能有一条默认策略。
- 默认策略的 `countryCodes` 必须是空数组。
- 普通策略至少选择一个国家。
- 同一渠道不同策略的国家不能重叠。
- 国家码统一转换为两位大写编码。
- `enabledCapabilities` 必须是渠道包能力的子集。
- `availableLoginTypes` 和 `loginMethods` 均只返回 `ThirdPlatformType.code` 数字字符串，不返回登录方式名称。
- `loginMethods` 只能使用配置中心当前选中的登录类型。
- `agreementGroupIds` 必须属于当前游戏。
- 保存接口按完整策略列表处理，请求中不存在的旧策略删除。

七、数据库设计

采用“同一张模块配置表，每个模块一条记录”的方案。

不把所有模块塞进同一条 JSON，也不为 `payment`、`login`、`agreement`、`thirdPartyData` 等分别建表。

一、数据结构

代码块

```
1  sdk_module_config
2  └─ appId + moduleCode 唯一
3      └─ payment          一条 JSON
4      └─ login            一条 JSON
5      └─ agreement        一条 JSON
6      └─ thirdPartyData   一条 JSON
7      └─ advertising      一条 JSON
```

这样兼顾：

- 模块间相互独立。
- 新增模块不需要新建表。
- 新增模块参数不需要修改数据库字段。
- 单字段、非全量更新可以在后端进行 JSON 合并。
- 登录方式、协议分组这类动态数组也比较适合 JSON。
- 不同模块同时保存时，不会修改同一条数据库记录。

二、 sdk_module_config 表

代码块

```
1 CREATE TABLE sdk_module_config (  
2     id BIGINT NOT NULL PRIMARY KEY,  
3     app_id BIGINT NOT NULL COMMENT '游戏ID',  
4     module_code VARCHAR(64) NOT NULL COMMENT '模块编码',  
5     config_json JSON NOT NULL COMMENT '模块完整配置JSON',  
6     schema_version INT NOT NULL DEFAULT 1 COMMENT 'JSON结构版本',  
7     data_version BIGINT NOT NULL DEFAULT 1 COMMENT '乐观锁版本',  
8     configured TINYINT NOT NULL DEFAULT 0 COMMENT '配置是否完整',  
9     missing_fields_json JSON NULL COMMENT '缺少的必填字段',  
10    created_by BIGINT NULL,  
11    updated_by BIGINT NULL,  
12    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
13    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP  
14             ON UPDATE CURRENT_TIMESTAMP,  
15  
16    UNIQUE KEY uk_app_module (app_id, module_code),  
17    KEY idx_app_id (app_id)  
18 ) COMMENT='SDK模块配置';
```

字段含义：

字段	说明
----	----

app_id	游戏ID
module_code	payment/login/agreement/thirdPartyData/advertising
config_json	当前模块的完整配置
schema_version	模块JSON结构发生升级时使用
data_version	防止并发修改互相覆盖
configured	当前模块是否满足完整配置要求
missing_fields_json	缺少的必填字段路径

三、局部更新实现方式

接口传递的 `config` 是补丁，不是完整配置。

例如只修改 Google 登录是否选中：

代码块

```
1 {
2   "appId": "791251136341225472",
3   "moduleCode": "login",
4   "config": {
5     "methods": [
6       {
7         "type": "3",
8         "selected": false
9       }
10    ]
11  }
12 }
```

后端处理流程：

```
1 根据 appId + moduleCode 查询原配置
2  → 对模块补丁进行参数校验
3  → 按模块规则合并JSON
4  → 校验合并后的完整配置
5  → 更新config_json
6  → data_version + 1
7  → 计算configured和missingFields
8  → 同步已有渠道配置
```

合并规则：

- 请求未出现的字段：保持原值。
- 普通对象：递归合并。
- `null`：清空字段。
- 空字符串：保存为空字符串。
- 登录方式数组：按 `type` 合并。
- 协议分组数组：按 `groupId` 合并。
- 三方数据数组：按 `type` 合并。
- Firebase 不允许通过普通保存接口修改。
- Google 支付不允许通过普通保存接口修改。

不直接使用数据库 `JSON_MERGE_PATCH` 处理全部逻辑，因为登录方式和协议分组是数组，需要按照业务主键合并。应该在 Java 服务层完成合并。

四、两个 PUT 接口如何更新

Google支付

`PUT /uploadGooglePaymentConfig` 只修改：

代码块

```
1 payment.channels.google
```

必须保留：

代码块

```
1 payment.notifyUrl
2 payment.channels.apple
3 payment.channels.oneStore
```

最终模块配置：

代码块

```
1 {
2   "notifyUrl": "https://api.example.com/payment/notify",
3   "channels": {
4     "apple": {
5       "pay_secret": "apple-secret"
6     },
7     "google": {
8       "pay_secret": "{完整JSON字符串}",
9       "serverConfigFile": {
10        "fileId": "9000001",
11        "fileName": "google-payment-config.json",
12        "fileUrl": "https://bucket.example.com/google-payment-config.json"
13      }
14    },
15    "oneStore": {
16      "licenseKey": "xxx",
17      "clientId": "xxx"
18    }
19  }
20 }
```

Firestore

PUT /uploadFirebaseConfig 根据 platform 只修改一个路径：

代码块

```
1 android → thirdPartyData.firebase.config.androidFile
2 ios     → thirdPartyData.firebase.config.iosFile
```

例如上传 Android 文件后：

代码块

```
1 {
2   "type": "firebase",
3   "config": {
4     "androidFile": {
5       "fileName": "google-services.json",
6       "fileUrl": "https://bucket.example.com/google-services.json"
7     },
8     "iosFile": {
9       "fileName": "GoogleService-Info.plist",
10      "fileUrl": "https://bucket.example.com/GoogleService-Info.plist"
11    }
12  }
13 }
```

更新 Android 时必须保留 iOS 原值。

五、上传文件表需要单独保留

文件本身可以使用独立表管理，不要只在模块 JSON 中保存文件信息。

代码块

```
1 CREATE TABLE sdk_config_file (
```



```

2      id      BIGINT NOT NULL PRIMARY KEY,
3      app_id  BIGINT NOT NULL,
4      module_code VARCHAR(64) NOT NULL,
5      usage_code VARCHAR(64) NOT NULL,
6      platform VARCHAR(16) NULL COMMENT 'android/ios',
7      file_name VARCHAR(255) NOT NULL,
8      file_url  VARCHAR(1000) NOT NULL,
9      object_key VARCHAR(500) NOT NULL,
10     content_type VARCHAR(128) NULL,
11     file_size    BIGINT NULL,
12     sha256       VARCHAR(64) NULL,
13     status       TINYINT NOT NULL DEFAULT 1 COMMENT '1有效 0已替换',
14     created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
15     updated_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
16     ON UPDATE CURRENT_TIMESTAMP,
17
18     KEY idx_app_usage (app_id, module_code, usage_code),
19     KEY idx_app_platform (app_id, module_code, platform)
20 );

```

用途包括：

- 记录文件上传后的地址。
- 追踪历史上传记录。
- 新文件保存失败时删除对象存储文件。
- 新文件替换成功后将旧记录标记为已替换。
- 模块配置中只保留页面真正需要的数据。

Firebase 模块 JSON 只保存 `fileName`、`fileUrl`；Google 支付模块还需要保存解析得到的完整 JSON 字符串。

六、需要单独建表的配置

不是所有内容都适合放入 `sdk_module_config`：

配置	存储建议
支付、登录、协议、三方数据、广告变现	<code>sdk_module_config</code> ，每个模块一条JSON
KWS参数、年龄阈值	继续使用 <code>app_parent_config</code>
上传文件记录	<code>sdk_config_file</code>
渠道基本配置和开关	现有渠道配置表
运营发行策略	单独的策略表

运营策略有独立 `strategyId`，支持多条记录、默认策略、国家冲突校验和删除，因此不建议存入模块 JSON。

代码块

```

1  CREATE TABLE sdk_operation_strategy (
2      id BIGINT NOT NULL PRIMARY KEY,
3      app_id BIGINT NOT NULL,
4      channel_id BIGINT NOT NULL,
5      strategy_name VARCHAR(128) NOT NULL,
6      is_default TINYINT NOT NULL DEFAULT 0,
7      country_codes JSON NOT NULL COMMENT '国家码数组',
8      enabled_capabilities JSON NOT NULL,
9      login_methods JSON NOT NULL,
10     agreement_group_ids JSON NOT NULL,
11     sort_no INT NOT NULL DEFAULT 0,
12     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
13     updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
14             ON UPDATE CURRENT_TIMESTAMP,
15
16     KEY idx_channel (app_id, channel_id)
17 );

```

最终就是：

一张 `sdk_module_config` 表统一承载模块配置，但每个模块独占一条记录；文件、KWS和运营策略等有独立生命周期的数据使用专用表。

这种结构最适合当前接口文档中的局部更新和后续参数扩展。